```
clear
clc
%profile on

cfg = config.getConfig();
cfg.path
cfg.filenames
```

Load

```
for i = 1:numel(cfg.filenames)
file = cfg.filenames(i)
tgt = fullfile(cfg.path, file + ".csv");
assert(isfile(tgt), "Cannot find source csv: %s", tgt);
dest = fullfile(pwd+"\outputs\", file + ".pdf");
fprintf("Processing %s\n to %s\n", tgt,dest);
TimeTemp = io.loadTimeTemp(tgt);
```

Timing analysis

```
[dt,dt_stats] = preprocessing.computeTimingJitter(TimeTemp);
plot.plot_dt_IntersampleInterval(dt) %figure
plot.plot_dt_boxchart(dt) %figure 1
plot.plot_dt_avar(dt_stats) %Figure 2
plot.plot_dt_percentile(dt_stats) %figure 3
```

Signal preprocessing

```
[robustAvgSmoothGauss,robustAvg] = preprocessing.computeRobustAverage( ...
    TimeTemp{:,cfg.sensorNames}, ...
    TimeTemp{:,"now"}, ...
    cfg);

plot(TimeTemp{:,"now"},robustAvgSmoothGauss); %Figure 4
xlabel("Time");ylabel("Temperature");title("Smoothed average")

noise =
preprocessing.computeNoise(TimeTemp{:,cfg.sensorNames},robustAvgSmoothGauss);

combinedSignal = [robustAvgSmoothGauss-mean(robustAvgSmoothGauss),noise];
signalNames = ["zeroed_rAveSG",cfg.sensorNames];
```

Full-record fingerprint

```
datafeatures = features.extractFingerprint(combinedSignal,signalNames);

[coeff,score,latent,tsquared,explained,mu] = analysis.runPCA(datafeatures);
%plot.plotPCAscore([score(:,1) score(:,2)],[explained(1) explained(2)]) %figure 5
```

```
%Rolling on the data
windowLength = 600;%maxpts = 600; %seconds of window
windowStep = 240;
windowOffsets = 0:windowLength-1;%windowpoints = 0:maxpts-1;
lastStart = size(combinedSignal,1)-windowLength+1;%seconds=height(combinedSignal)-
maxpts;
windowStarts = 1:windowStep:lastStart;%slices = 1:maxpts/2:seconds;
nWindows = numel(windowStarts); %nWindows=length(slices);
Fingerprints = cell(nWindows,1);

for i=1:nWindows
    t=windowStarts(i);
    f = features.extractFingerprint(combinedSignal(t+windowOffsets,:),signalNames);
    %Fingerprints{i}.Table = f;
    Fingerprints{i}.Matrix = table2array(removevars(f,"SensorName"));
    if i == 1
        rollingFeatureTemplate = f;
    end
end
figure
hold on
for i=1:nWindows
    [coeff,score,latent,tsquared,explained,mu] =
analysis.runPCA(Fingerprints{i}.Matrix);
    XY = score(:,1:2);
    XY = XY - mean(XY,1);% Center layer
    vRef = XY(1,:);% Reference is zeroed_rAveSG, row 1
    theta = atan2(vRef(2),vRef(1));% Angle of reference
    R = [cos(-theta) -sin(-theta);sin(-theta) cos(-theta)];% Rotate reference onto
+X axis
    XYaligned = XY * R';

    % Scale reference to radius 100
    refDistance = norm(XYaligned(1,:));

    scale = 100 / refDistance;

    XYscaled = XYaligned * scale;

    % Sanity checks
    assert(XYscaled(1,1) >= 100 - 1e-8)
    assert(abs(XYscaled(1,2)) < 1e-8)

    FRFpt = scatter(XYscaled(:,1), XYscaled(:,2));%,i*ones(size(XYscaled,1)))
end
xlabel("ScoreX");ylabel("ScoreY");%zlabel("Window");
hold off
%FRFpt.view(0,90) %Az,El
```

Temporal fingerprints

```
%flatten the vectors for processing
FeatureMatrixRaw = zeros(nWindows,numel(Fingerprints{1}.Matrix));

for i = 1:nWindows
    FeatureMatrixRaw(i,:) = Fingerprints{i}.Matrix(:);
end
size(FeatureMatrixRaw);
```

Build flattened rolling feature names

```
featureNames =
string(removevars(rollingFeatureTemplate,"SensorName").Properties.VariableNames);
sensorNames = string(rollingFeatureTemplate.SensorName);
flatFeatureNames = strings(numel(sensorNames) * numel(featureNames),1);
idx = 1;

for feature = 1:numel(featureNames)
    for sensor = 1:numel(sensorNames)
        flatFeatureNames(idx) = sensorNames(sensor) + "__" + featureNames(feature);
        idx = idx + 1;
    end
end
assert( ...
    numel(flatFeatureNames) == size(FeatureMatrixRaw,2), ...
    "Feature names do not match flattened fingerprint width.");
```

Validate temporal feature matrix

```
featureStd = std(FeatureMatrixRaw,0,1);

constantCols = featureStd == 0;
nonFiniteCols = any(~isfinite(FeatureMatrixRaw),1);

fprintf("Constant feature dimensions: %d\n",nnz(constantCols));
fprintf("Nonfinite feature dimensions: %d\n",nnz(nonFiniteCols));

validCols = ~constantCols & ~nonFiniteCols;

FeatureMatrix = FeatureMatrixRaw(:,validCols);

fprintf("Accepted: %d / %d feature dimensions\n", ...
    width(FeatureMatrix), ...
    width(FeatureMatrixRaw));
acceptedFeatureNames = flatFeatureNames(validCols);
rejectedFeatureNames = flatFeatureNames(~validCols);
```

Deltas

deltaRaw: absolute fingerprint difference in the original feature scales

```
deltaRaw = squareform(pdist(FeatureMatrix,'euclidean'));
% deltaZ: standardized fingerprint difference, where high-magnitude features don't
automatically dominate
deltaZ = squareform(pdist(zscore(FeatureMatrix),'euclidean'));

figure
imagesc(deltaRaw) %Figure 6
axis image
colorbar
xlabel('Window');ylabel('Window');title('Fingerprint Distance Matrix delta')

figure
imagesc(deltaZ) %figure 7
axis image
colorbar
xlabel('Window');ylabel('Window');title('Fingerprint Distance Matrix deltaZ')

% Drift Measurement

nLags = size(deltaRaw,1)-1;
driftRaw = zeros(nLags,1);
driftZ = zeros(nLags,1);
for k = 1:nLags
    driftRaw(k)=mean(diag(deltaRaw,k));
    driftZ(k)=mean(diag(deltaZ,k));
end

lagSamples = (1:nLags) * windowStep;
lagSeconds = lagSamples / cfg.sampleRate;
figure
plot(lagSeconds,driftRaw,'o-')
xlabel("Temporal Separation (s)")
ylabel("Mean Fingerprint Distance")
title("Fingerprint Drift vs Temporal Separation")
grid on

figure
plot(lagSeconds,driftZ,'o-')
xlabel("Temporal Separation (s)")
ylabel("Normalized Mean Fingerprint Distance")
title("Normalized Fingerprint Drift vs Temporal Separation")
grid on
nPairs = nWindows - (1:nLags);

figure
semilogx(lagSeconds,nPairs)
```

```
xlabel("Temporal Separation (s)")
ylabel("Number of Window Pairs")
title("Support for Drift Estimate")
grid on
```

MultiDimentional Scale

```
tic
Yraw = mdscale(deltaRaw,3);
toc

figure
scatter3(Yraw(:,1),Yraw(:,2),Yraw(:,3),20,1:size(Yraw,1),'filled')
xlabel("MDS 1");ylabel("MDS 2");zlabel("MDS 3");colorbar;
title("Fingerprint Evolution")
grid on
clear Yraw

tic
Yz   = mdscale(deltaZ,3);
toc

figure
scatter3(Yz(:,1),Yz(:,2),Yz(:,3),20,1:size(Yz,1),'filled')
xlabel("MDS 1");ylabel("MDS 2");zlabel("MDS 3");colorbar;
title("Normalized Fingerprint Evolution")
grid on
clear Yz
```

build a Directed Acyclic Graph

```
% figure %figure 10
% upperDistances = deltaRaw(triu(true(size(deltaRaw)),1));
% threshold = prctile(upperDistances,10);
%
% A = deltaRaw < threshold;
% A = triu(A,1);
% G = digraph(A);
% plot(G)
% title("graph of A")
```

Detect regime changes

```
figure
adjacentDistanceRaw = diag(deltaRaw,1);
plot(adjacentDistanceRaw)
xlabel("Window");ylabel("Adjacent Fingerprint Distance");
title("Fingerprint Change Between Adjacent Windows");
grid on
```

```
figure
adjDistanceZ = diag(deltaZ,1);
plot(adjDistanceZ);
xlabel("Window");ylabel("Standardized AdjacentFingerprint Distance");
title("Standardized Fingerprint Change Between Adjacent Windows")
grid on

figure
meanDistanceRaw = mean(deltaRaw,2);
plot(meanDistanceRaw)
xlabel("Window");
ylabel("Mean Global distance to all windows");
title("Global Fingerprint Difference by Window");
grid on

figure
meanDistanceZ = mean(deltaZ,2);
plot(meanDistanceZ)
xlabel("Window");
ylabel("Mean Normalized distance to all windows");
title("Normalized Global Fingerprint Difference by Window");
grid on


%profile viewer
source = fullfile(pwd,"main.mlx");
assert(isfile(source), "Cannot find source Live Script: %s", source);
export(source,dest);
end
```